Project HiveMind: A System Design for Decentralized Sparse Mixture-of-Experts Training with DiLoCo-Style Outer Optimization on 8 GB Consumer GPUs

Integration of Low-Communication Training Methods, Controlled Experimental Evaluations, and Lean 4 Bookkeeping Lemmas

Xavier Callens*
Project HiveMind Open-Source Contributors

October 23, 2026 — Version 3.2.0 (revised with analytical proofs, necessity counterexamples, a Decoupled DiLoCo quorum syncer, and 120 Lean 4 lemmas)

Abstract

Training and serving frontier Large Language Models (LLMs) are increasingly monopolized by hyperscale datacenters due to two fundamental systems bottlenecks: the **VRAM Wall** (a single 8 GB–12 GB consumer GPU cannot hold a 70B+ dense model or its FP32 AdamW optimizer states) and the **Bandwidth Wall** (standard synchronous All-Reduce Distributed Data Parallelism saturates 10–50 Mbps residential internet links instantly). Yet the global PC gaming and open-source developer community possesses millions of idle NVIDIA RTX 20/30/40-series, AMD ROCm, and Apple Metal GPUs alongside terabytes of NVMe flash storage.

We describe **Project HiveMind**, an open-source decentralized training and inference system, and report what has and has not yet been validated. The system combines **Branch-Train-Mix (BTM) sparse MoE upcycling**, **weight-tied looped recurrence**, and **DiLoCo-style outer optimization**, with an emphasis on fitting a strict 8 GB GPU memory budget. Workers stream open-weight backbones (`Qwen2.5`) into 4-bit NF4 and train QLoRA adapters; outer pseudo-gradients can be compressed with a DCT + 2-bit centroid scheme adapted from DeMo [18] and SparseLoCo [19]; the outer optimizer supports a CPU-resident flat-bucket state (after `prime-diloco` [7]) and the staleness gate of CGAD [14]; converged branches are upcycled into a sparse MoE with optional Iso-CTS [15] or TSV [17] attention merging and FoMoE-style partial expert replication [10].

Empirical and formal evaluation. We evaluate the system at three levels, separating verified properties from open multi-node questions: (i) Ten *unit-level implementation checks* (`H1`–`H10`, `artifacts/architecture_hypothesis_verification_report.json`) verify that each component satisfies its mathematical specification on controlled tensors, including non-trivial fixed-point halting ($\epsilon_{\text{fp}} = 0.65 \Rightarrow 3.05/4.0$ mean recurrent steps) and single-round vs. multi-step EF21 error-feedback conservation; an eleventh check (`H11`, `artifacts/decoupled_diloco_quorum_check.json`) covers a Decoupled DiLoCo syncer [5] with minimum quorum, adaptive grace window and token-weighted Radial-Directional Averaging, which restores the common outer-gradient norm that direct averaging shrinks by $1/\sqrt{M}$ (2.83× at $M = 8$). (ii) Four *controlled multi-regime experimental studies* (`E1`–`E4`, `artifacts/paper_v3_experiments_report.json`, Figures 1–4) measure held-out generalization across five synchronization regimes (showing that DeMo 2D-DCT + SparseLoCo 2-bit + EF21 achieves a **0.052%** held-out loss gap vs. uncompressed FP32 DiLoCo while cutting outer wire traffic by **12.49**× and **99.9**× vs. synchronous DDP, and that `MuLoCo` 5-step Newton-Schulz inner preconditioning slightly improves compressed held-out perplexity), outer convergence under severe straggler bursts ($\tau_w \geq 36$, where `PA-CGAD` reduces terminal MSE by **11.6**× relative to un-gated outer AdamW), fixed-point halting trade-offs across $\epsilon_{\text{fp}} \in [0.0, 1.10]$, and

*Project HiveMind (`callensxavier@gmail.com`). Code and all artifacts referenced in this paper: https://github.com/xaviercallens/HiveMind. The project is non-commercial; contributors receive no payment.

singular value spectra under `Iso-CTS` and `TSV` merging. (iii) A single-GPU proof of concept on an 8 GB NVIDIA RTX 2070 with `Qwen2.5-0.5B` trains four domain branches and upcycles them into a 4-expert top-2 MoE within **1,509.5 MB** peak VRAM, accompanied by **120** zero-`sorry` Lean 4 arithmetic and hypothesis-necessity lemmas (`lean4/HiveMindSpec.lean`). Section 9 details remaining multi-host WAN limitations.

Keywords: Decentralized AI · Sparse Mixture-of-Experts (MoE) · DiLoCo & OpenDiLoCo · FoMoE · Looped Transformers · DeMo DCT & SparseLoCo · MuLoCo Newton-Schulz · Byzantine Fault Tolerance · Lean 4

Contents

1 Introduction: Breaking the VRAM and Bandwidth Walls **4**
1.1 Architecture Overview & Summary of Contributions 4

2 Lean 4 Bookkeeping Lemmas: Scope and Limits **5**

3 Pillar I: Zero-Copy 4-Bit NF4 & DiLoCo-QLoRA Foundation **6**
3.1 Zero-Copy Meta-Device Streaming & 4-Bit NormalFloat Quantization 6
3.2 DiLoCo Inner Loop & Baseline Top-k + `float8_e4m3fn` Deltas 6

4 Pillar II: Byzantine Fault Tolerance Without 2× Redundancy **7**
4.1 10% Deterministic Micro-Checkpoint Spot-Checks & Hardware Circuit-Breaker . . 7
4.2 Pre-Krum Sanitization, Multi-Krum 2σ Cosine Filter & ZK-OPoI v4 8

5 Pillar III & Phase 2 Swarm Fabric: Factored BTM Upcycling & Geo-Rings **8**

6 Missing MoE Architecture: 128-Expert 3-Tier Routing, FoMoE, Skip-Attention & Looped Recurrence **8**
6.1 3-Tier 128-Expert Hierarchical MoE Router (`ThreeTierHierarchicalMoERouter`) 9
6.2 Federated Partial Expert Replication (`FoMoE`) & Ghost-Expert `Skip-Token` 9
6.3 `Skip-Attention` Speculative Prefetch & Cache-Aware Routing (`CAR`) 10
6.4 Weight-Tied `LoopedExpertBackbone`, Fixed-Point Halting & Truncated BPTT . . 10
6.5 Subspace-Aware Attention & Expert Merging: `Iso-CTS` & `TSV` 11

7 OpenDiLoCo & Google DeepMind Outer-Loop Ecosystem **11**
7.1 `OpenDiLoCo` & `prime-diloco`: Contiguous 1D CPU Flat-Bucket Outer State (`0 B` GPU VRAM) 12
7.2 Decoupled DiLoCo: Minimum Quorum, Adaptive Grace Window & Radial-Directional Averaging 12
7.3 Streaming DiLoCo Fragments & Cosine-Gated Adam-Decay (`PA-CGAD`) 13
7.4 `DeMo` Chunked 2D DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo` + `EF21` 14
7.5 `AutoLoCo`, `HeLoCo` & Inner-Loop `MuLoCo` Newton-Schulz Preconditioning 14
7.6 Analytical Theorems, Proofs, Hypothesis-Necessity Counterexamples & 4-Definitions Contract 15

8 Empirical Verification, Controlled Experiments & Visualizations **16**
8.1 Unit-Level Implementation Checks (`H1`–`H10`) 16
8.2 Controlled Experimental Evaluations (`E1`–`E4`) & Visualizations 16
8.2.1 Experiment E1: Held-Out Generalization & Wire Efficiency Across Five DiLoCo / MuLoCo Regimes 17
8.2.2 Experiment E2: `PA-CGAD` Outer Convergence Under Severe Asynchronous Straggler Bursts 18

8.2.3 Experiments E3 & E4: `LoopedMoE` Fixed-Point Halting Sweep, Subspace Spectra & Hardware Scaling 19
8.3 Single-GPU Proof of Concept on an 8 GB NVIDIA GeForce RTX 2070 & FoMoE Scaling 19

9 Limitations, Threats to Validity, and Future Multi-Host Benchmarks 20
9.1 Epistemic Stratification & 6-Layer Security/Hardness Attestation 21

1 Introduction: Breaking the VRAM and Bandwidth Walls

Modern frontier language models rely on massive parameter counts (70B to 1T+) trained over tightly coupled InfiniBand or NVLink fabrics (400–3,200 Gbps). Attempting to decentralize LLM training and inference across volunteer consumer gaming hardware—the modern equivalent of `SETI@home` [1] and `Folding@home`—confronts four structural barriers:

1. **The Consumer VRAM Wall:** The median gaming GPU on the Steam Hardware Survey (e.g., NVIDIA RTX 2070, RTX 3060, RTX 4060) provides **8 GB to 12 GB of GDDR6 VRAM**. On Windows 10/11 under the WDDM display driver model, approximately 1.0–1.5 GB is reserved by the operating system and desktop compositor, leaving a strict usable ceiling of $M_{\text{ceil}} \approx 6.5$ GB on an 8 GB card. A dense 7B model in FP16 with FP32 master weights and AdamW first/second moments requires > 56 GB of VRAM, causing immediate out-of-memory (OOM) failure.

2. **The Residential WAN Bandwidth & Straggler Wall:** Standard Distributed Data Parallel (DDP) training executes an All-Reduce over all gradients at every micro-step ($t = 1, 2, \dots$). Over residential fiber or cable connections (10–50 Mbps upload, 20–150 ms RTT), synchronizing gigabytes of gradients per second is physically impossible. Even under Distributed Low-Communication (`DiLoCo`) [2] training, asynchronous WAN stragglers inject stale pseudo-gradients that corrupt outer momentum and inflate outer Adam second moments [14].

3. **The Sparse MoE All-to-All & PCIe Thrashing Wall:** Scaling model capacity via Mixture-of-Experts (MoE) [30, 8, 6] traditionally requires either holding all E experts in GPU VRAM (violating the 8 GB ceiling when $E = 64$–128) or issuing synchronous token-level All-to-All RPCs and synchronous PCIe DMA loads that stall the GPU for 2–20 ms per layer.

4. **The Byzantine & Gamer Preemption Threat:** Volunteer consumer nodes include unstable GPU overclocks, bit-flips, Sybil botnets, and adversarial gradient poisoners. Classic BOINC mitigates this via 2× redundant workunit replication (wasting 50% of cluster FLOPs), while gamers demand < 1.5 s preemption latency whenever a game launches.

1.1 Architecture Overview & Summary of Contributions

Project HiveMind addresses these barriers with four design pillars (Sections 3–5), a next-generation **Hierarchical, Federated & Looped Sparse MoE architecture** (Section 6), and a unified **Google DeepMind DiLoCo + Prime Intellect OpenDiLoCo outer-loop stack** (Section 7):

- **Pillar 1 (Zero-Copy 4-Bit NF4 + MuLoCo-Preconditioned DiLoCo-QLoRA):** Meta-device skeleton initialization with streaming 4-bit NormalFloat (`NF4`) double quantization (`0 B` CPU heap duplication) paired with inner-loop **5-step quintic Newton-Schulz LoRA gradient orthogonalization** (Muon [23], as used inner-loop by MuLoCo [24, 25]) and **EF21 error-feedback wire compression** [26, 19, 18].

- **Pillar 2 (Byzantine Fault Tolerance Without 2× Replication):** Deterministic CUDA execution (`CUBLAS_WORKSPACE_CONFIG=:4096:8`) with SHA-256 micro-checkpoints every $s = 50$ steps enables 10% random segment replay within $\epsilon \leq 10^{-4}$ (20× fewer verification FLOPs than 2× BOINC), paired with **Multi-Krum 2σ cosine filtering** [29] and **ZK-OPoI v4 position-bound batch Merkle verification**.

- **Pillar 3 & Missing MoE Architecture (128-Expert 3-Tier Hierarchical MoE, `FoMoE`, `Skip-Attention` & `LoopedExpertBackbone`):** We extend Branch-Train-Mix (BTM)

[8] with (i) a **3-Tier** $4 \times 4 \times 8 = 128$**-Expert Hierarchical Router** (8.0× gating FLOP reduction), (ii) **Federated Partial Expert Replication (`FoMoE`)** [10] with **Ghost-Expert `Skip-Token`** local routing (3.46× WAN ring reduction, zero remote FFN RPCs), (iii) **`Skip-Attention` Speculative Prefetch + Cache-Aware Routing (`CAR`)** [11] (so far evaluated only in a synthetic unit check), (iv) **`Iso-CTS` & `TSV` Subspace Attention Merging** [15, 16, 17], and (v) a **Weight-Tied `LoopedExpertBackbone`** (`Prelude` → `RecurrentCore` $\times T \rightarrow$ `Coda`) [12, 13] with per-token fixed-point convergence exit and truncated BPTT.

- **Next-Generation OpenDiLoCo & Google DeepMind Outer-Loop Stack:** Building on Google DeepMind's `DiLoCo` [2], asynchronous Local-SGD [3], `Streaming DiLoCo` [4], `Scaling Laws for DiLoCo` [6], and `Decoupled DiLoCo` [5] with Prime Intellect's `OpenDiLoCo` / `prime-diloco` [7], we implement a **Contiguous 1D CPU Flat-Bucket Outer State** (`0 B` extra GPU VRAM), **Cosine-Gated Adam-Decay (`PA-CGAD`)** [14] with uniform staleness bias bound $\max_{\tau \geq 0} \tau\sigma(\tau) \leq \frac{1}{e\alpha}$, **`DeMo` + `SparseLoCo` + `PULSELoCo-style` 2-bit DCT wire compression** [18, 19, 20], **`AutoLoCo`** [22], **`HeLoCo`** [21], and a **Decoupled DiLoCo quorum syncer** (minimum quorum K, adaptive grace window $\xi_{\text{grace}} \leq \gamma\, \xi_{\text{slack}}$, token-weighted Radial-Directional Averaging; Section 7.2).
- **Honest scope statement:** each pillar is implemented and unit-tested, but the end-to-end system has not yet been evaluated across multiple machines or against a data-parallel baseline (Section 9).

2 Lean 4 Bookkeeping Lemmas: Scope and Limits

We maintain a Lean 4 [31] file of arithmetic and necessity lemmas that accompany the design (`lean4/HiveMindSpec.lean`, compiled with `leanprover/lean4:v4.33.0-rc2`). All **120** lemmas compile with no `sorry`, no `admit`, and no user axioms (`lake build`, `#print axioms`).

What these lemmas establish and what they do *not* establish. Every statement is over $\mathbb{N}$ or $\mathbb{Z}$; there is no real-valued functional analysis, no stochastic convergence theory, and no machine-checked refinement link between a Lean statement and the Python/CUDA runtime. Rather, the 120 lemmas serve as machine-checked arithmetic, hypothesis-necessity counterexamples, and non-vacuity invariants: e.g., that contiguous flat-bucket slice offsets partition the outer parameter buffer without overlap (Thm. 101), that staggered fragment streaming strictly reduces per-round peak serialization bytes (Thm. 102), that the piecewise PA-CGAD cutoff rule $\tau_{\text{eff}} \geq \tau_{\text{cut}}$ freezes the discrete second-moment accumulator v_{prev} strictly below the un-gated polluted value $v_{\text{prev}} + g_{\text{sq}}$ (Thm. 103), that a 4-tensor $[256, 16]$ 2-bit DCT package (820 B) clears both the 16× 16-bit and 32× FP32 compression gates (Thm. 105), that inward singular-value pulls $\delta > 0$ strictly contract the spectral spread $(s_{\max} - \delta) - (s_{\min} + \delta) < s_{\max} - s_{\min}$ (Thm. 106), that EF21 error-feedback residuals telescope across consecutive outer rounds (Thm. 107), and that dropping EF21 residual carry (Thm. 113), omitting the PA-CGAD staleness cutoff (Thm. 114), or violating the Multi-Krum $n \geq 2f + 3$ quorum bound (Thm. 115) strictly breaks each respective invariant, while `InhabitedRTX2070SwarmPackage` is provably non-empty (Thm. 116). Season 14 (Thms. 117–120) adds the Decoupled DiLoCo bookkeeping of Section 7.2: a grace window $\gamma\, \xi_{\text{slack}}$ with $\gamma < 1$ is strictly below any positive slack (Thm. 117), an exhausted compute budget yields a zero grace window (Thm. 118), the token-weighted learner weight is monotone in tokens and antitone in steps (Thm. 119), and the admitted set has size in $[K, M]$ while the degenerate $K = 0$ rule admits nobody (Thm. 120). Continuous-time optimization convergence, the real-valued calculus proof of $\sup_{\tau \geq 0} \tau e^{-\alpha\tau} = \frac{1}{e\alpha}$ (proved analytically in Theorem 7.2), and adaptive Byzantine resilience are *not* formalized in Lean 4.

Listing 1: Excerpt from `lean4/HiveMindSpec.lean` (Season 12: OpenDiLoCo Flat-Bucket, PA-CGAD Piecewise Cutoff, DeMo-SparseLoCo 2-Bit Gate, MuLoCo Spread Contraction, and EF21 Telescoping Sum).

```
namespace HiveMindSpec.Season12_OpenDiLoCoAndMissingMoEArchitectureHypotheses

theorem opendiloco_flat_bucket_partition_conserves_numel (n1 n2 n3 : Nat) :
    (n1 - 0) + ((n1 + n2) - n1) + ((n1 + n2 + n3) - (n1 + n2)) = n1 + n2 + n3 :=
      by
  omega

theorem pacgad_staleness_cutoff_preserves_second_moment
    (v_prev g_sq tau_eff tau_cut : Nat)
    (h_cutoff : tau_cut <= tau_eff) (h_gsq : 0 < g_sq) :
    (if tau_eff < tau_cut then v_prev + g_sq else v_prev) = v_prev /\
      v_prev < v_prev + g_sq := by
  have h_not_lt : Not (tau_eff < tau_cut) := Nat.not_lt.mpr h_cutoff
  simp [h_not_lt]; omega

theorem demo_sparseloco_2bit_package_exceeds_16x_compression :
    16 * (4 * (21 + 82 * 2 + 8 + 12)) <= 4 * 4096 * 2 /\
      32 * (4 * (21 + 82 * 2 + 8 + 12)) <= 4 * 4096 * 4 := by
  decide

theorem muloco_newton_schulz_condition_contraction
    (s_max s_min delta : Nat) (h_delta : 0 < delta)
    (h_order : s_min + delta <= s_max - delta) :
    (s_max - delta) - (s_min + delta) < s_max - s_min := by
  omega

theorem ef21_multistep_telescoping_sum_conservation
    (g1 g2 c1 r1 c2 r2 : Int)
    (h_step1 : c1 + r1 = g1) (h_step2 : c2 + r2 = g2 + r1) :
    (c1 + c2) + r2 = g1 + g2 := by
  omega
end HiveMindSpec.Season12_OpenDiLoCoAndMissingMoEArchitectureHypotheses
```

3 Pillar I: Zero-Copy 4-Bit NF4 & DiLoCo-QLoRA Foundation

3.1 Zero-Copy Meta-Device Streaming & 4-Bit NormalFloat Quantization

Standard model loaders instantiate a dense FP16/FP32 weight tree in system RAM before transferring tensors to the GPU, causing host pagefile exhaustion on 16 GB RAM gaming PCs. In Project HiveMind (`src/hivemind/core/quantization.py`), we initialize the causal Transformer architecture on PyTorch's zero-byte `meta` device (`init_empty_weights`) and stream each tensor slice directly from memory-mapped `safetensors` shards onto `cuda:0`, immediately quantizing linear projections $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ into 4-bit NormalFloat (`NF4`) [9] with block size $B = 64$:

$$\hat{W}_{\text{NF4}}(i) = \arg\min_{j \in \{0,\dots,15\}} \left| \frac{W_0(i)}{s_b} - q_j \right|, \qquad s_b = \max_{i \in \text{block } b} |W_0(i)|, \tag{1}$$

where $q_0, \dots, q_{15} \in [-1, 1]$ are the information-theoretically optimal quantiles of $\mathcal{N}(0, 1)$, packed two 4-bit indices per `uint8` byte. Block scales s_b are further compressed via 8-bit double quantization, yielding an effective footprint of 4.125 bits/parameter (**Theorem 1**). On `Qwen/Qwen2.5-0.5B`, the entire frozen backbone occupies just **175.96 MB** of VRAM with **0 bytes** of intermediate CPU heap duplication.

3.2 DiLoCo Inner Loop & Baseline Top-k + `float8_e4m3fn` Deltas

Each worker $k \in \{1, \dots, K\}$ assigned to domain branch $d \in \mathcal{D}$ freezes $\hat{W}_{\text{NF4}}$ and attaches trainable low-rank adapters $\theta_k = \{A_k, B_k\}$ of rank $r \in \{8, 16, 32, 64\}$ across all attention and SwiGLU

Table 1: Inventory of the 120 Lean 4 lemmas in `lean4/HiveMindSpec.lean`. Every lemma compiles with zero `sorry` and verifies discrete arithmetic, necessity counterexamples, and resource bounds over $\mathbb{N}/\mathbb{Z}$.

Season / Namespace	Theorem Range	Count	Core Mathematical Invariants Proved	Status
Pillars 1–4 & Phase 2	Thms. 1–14	14	NF4 4.125 b/w, 8 GB RTX 2070 ceiling, 10% Spot-Check (20× savings), Multi-Krum, Top-2 MoE, Async DMA 0 ms stall	compiles
Season 3 (Prefix Trie & 0-RTT)	Thms. 15–26	12	Prefix-Trie FLOP savings, RFC 9001 QUIC 0-RTT monotonic anti-replay, speculative rejection sampling invariance	compiles
Season 4 (DAO & Super-Ring)	Thms. 27–32	6	Quadratic voting cost, Steamworks anti-Sybil bound, Continental Super-Ring hop bound, SVD rank truncation, OPoI Merkle depth	compiles
Season 5 (Strategy & 64-MoE)	Thms. 33–40	8	TIES sign consensus, Interactive Bisection $\lceil \log_2 L \rceil$, 2-Level 64-Expert Router (4×), FP8 Paged KV-Cache, Fisher SVD budget	compiles
Season 6 (EF21 & 128-MoE)	Thms. 41–44, 57–69	17	3-Tier 128-Expert Router (8× FLOP savings), EF21 error-feedback contraction, energy split, UCB1 bandit optimality	compiles
Seasons 7–9 (Hardening & Safety)	Thms. 45–56, 70–77	20	Adaptive median norm clipping, staleness window rejection, thermal hysteresis deadband, WDDM TDR preemption, ZK-OPoI v4	compiles
Season 10 (Next-Gen DiLoCo)	Thms. 78–90	13	PA-CGAD cosine cutoff, FoMoE ring savings & Skip-Token, PULSELoCo half-ULP gate, Skip-Attention CAR, IsoLoCo, AutoLoCo, HeLoCo, `prime-diloco` 0 B VRAM, SparseLoCo 2-bit, LoopedMoE, MuLoCo	compiles
Season 11 (Looped FP & IsoSparse)	Thms. 91–100	10	Fixed-point recurrence contraction, early-exit FLOP savings, 3× terminal KV sharing, multi-wave FoMoE VRAM & payload bounds	compiles
Season 12 (OpenDiLoCo & H1–H10)	Thms. 101–112	12	Flat-bucket slice partition, fragment byte count, piecewise PA-CGAD cutoff, 16×/32× 2-bit DCT gate, MuLoCo spread contraction, EF21 telescoping sum, TBPTT memory, Skip-Token FLOPs	compiles
Season 13 (Necessity & Inhabitation)	Thms. 113–116	4	Disabled-EF21 mass loss counterexample, missing PA-CGAD cutoff variance inflation counterexample, violated Multi-Krum quorum counterexample, `Nonempty InhabitedRTX2070SwarmPackage` witness	compiles
Season 14 (Decoupled DiLoCo Quorum)	Thms. 117–120	4	Grace window $\gamma\,\xi_{\text{slack}}$ strictly below positive slack for $\gamma < 1$, exhausted slack $\Rightarrow$ zero grace, token-weight monotonicity, quorum admitted-count bounds $K \le \lvert\mathcal{A}\rvert \le M$	compiles
Total Formal Specification	**Theorems 1–120**	**120**	Arithmetic over $\mathbb{N}/\mathbb{Z}$; standard Lean 4 axioms (`propext`, `Quot.sound`, `Classical.choice`)	120 / 120 compile

projections ($\{W_q, W_k, W_v, W_o, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}\}$):

$$h = \text{Dequant}_{\text{FP16}}(\hat{W}_{\text{NF4}})x + \frac{\alpha}{r} B_k A_k x, \qquad A_k \in \mathbb{R}^{r \times d_{\text{in}}},\ B_k \in \mathbb{R}^{d_{\text{out}} \times r}. \tag{2}$$

Worker k executes H local inner steps on its Content-Addressed Hugging Face shard (`cid://sha256/...`) with **zero network communication** and computes its outer pseudo-gradient delta $\Delta_k^{(t)} = \theta^{(t-1)} - \theta_k^{(t,H)}$. In baseline mode (`compression_mode="topk_fp8"`), retaining Top-k ($\rho = 0.20$) entries quantized to `float8_e4m3fn` compresses a 16.78 MB rank-16 QLoRA delta into **5.04 MB** (3.33× wire reduction, or $> 1,666\times$ total bandwidth reduction at $H = 500$, **Theorem 3**). Section 7 details how Season 8 upgrades this wire layer with **DeMo 2D chunked DCT + 2-bit SparseLoCo + PULSELoCo** to achieve **39.96× package compression**.

4 Pillar II: Byzantine Fault Tolerance Without 2× Redundancy

4.1 10% Deterministic Micro-Checkpoint Spot-Checks & Hardware Circuit-Breaker

To verify untrusted consumer GPUs without running every workunit twice, we enforce deterministic CUDA execution (`CUBLAS_WORKSPACE_CONFIG=:4096:8`,

`torch.use_deterministic_algorithms(True)`, and deterministic data-loader seeding derived from `prev_block_hash`). During its H inner steps, worker k records a cryptographic micro-checkpoint hash every s steps ($M = H/s = 10$ segments):

$$h_{k,m} = \text{SHA-256}\left(\theta_k^{(t,m\cdot s)}\right), \qquad m \in \{0, 1, \ldots, M\}. \tag{3}$$

The `DeterministicSpotCheckValidator` (`src/hivemind/boinc/validator.py`) selects a single random segment $m^* \sim \mathcal{U}\{0, \ldots, M-1\}$ and replays s steps from $\theta_k^{(t,m^*s)}$. The submission is accepted iff $\|\theta_{\text{validator}}^{(t,(m^*+1)s)} - \theta_k^{(t,(m^*+1)s)}\|_\infty \leq \epsilon = 10^{-4}$ (**Theorems 4–5**). Concurrently, the `GPUExecutionCircuitBreaker` (`src/hivemind/security/gpu_execution_circuit_breaker.py`) enforces thermal hysteresis deadbands (83°C throttle / 88°C abort), Windows WDDM TDR step-latency watchdogs (< 1.5 s), and NaN loss early abort (**Theorems 49–52**).

4.2 Pre-Krum Sanitization, Multi-Krum 2σ Cosine Filter & ZK-OPoI v4

Before outer aggregation, `FederatedLearningSecurityGuard` (`src/hivemind/security/federated_learning_hardening.py`) quarantines non-finite or free-rider zero-gradient submissions, applies Multi-Krum neighborhood selection ($k = n-f-2 \geq f+1$, **Theorem 6**) with a 2σ median pairwise cosine similarity gate, and clips surviving deltas via adaptive median-L_2 and coordinate-L_∞ neurotoxin bounds (**Theorem 45**). For decentralized inference, **ZK-OPoI v4** (`src/hivemind/swarm/swarm_fabric.py`) binds per-token activation commitments to their exact batch index, layer index, and token position inside a SHA-256 Merkle tree with interactive $\lceil \log_2 L \rceil$ bisection dispute resolution (**Theorems 35–36, 70–77**).

5 Pillar III & Phase 2 Swarm Fabric: Factored BTM Upcycling & Geo-Rings

When E domain branches converge, `upcycle_branches_to_sparse_moe()` (`src/hivemind/core/btm_upcycle.py`) converts them into a unified **Factored Sparse MoE** where all E SwiGLU experts at layer ℓ share the single frozen 4-bit NF4 base FFN $(\hat{W}_{\text{gate}}, \hat{W}_{\text{up}}, \hat{W}_{\text{down}})$ and keep only their lightweight FP16 LoRA adapters (A_e, B_e) in VRAM:

$$\begin{aligned} E_{\ell,e}(x) = \left(\hat{W}_{\text{down}} + \tfrac{\alpha}{r} B_{\text{down},e} A_{\text{down},e}\right) \Big[\text{SiLU}\left(\hat{W}_{\text{gate}} x + \tfrac{\alpha}{r} B_{\text{gate},e} A_{\text{gate},e} x\right) \\ \odot \left(\hat{W}_{\text{up}} x + \tfrac{\alpha}{r} B_{\text{up},e} A_{\text{up},e} x\right) \Big]. \end{aligned} \tag{4}$$

Across WAN nodes, the **Phase 2 Swarm Fabric** (`src/hivemind/swarm/swarm_fabric.py`) provides: (M1) Kademlia XOR < 20 ms Regional Geo-Rings with RFC 8489/8656 STUN/TURN NAT traversal and RFC 9001 QUIC 0-RTT anti-replay (**Theorems 11, 16–24**); (M2) 3-tier NVMe → Pinned Host RAM → CUDA double-buffered asynchronous DMA streaming (**Theorem 12**); (M3) Prefix-Trie deduplicated Eagle/Medusa token-tree speculation ($\gamma = 6$) with Triton `float8_e4m3fn` hidden-state compression (**Theorems 13, 15**); and (M4) an OpenAI-compatible `/v1/chat/completions` gateway.

6 Missing MoE Architecture: 128-Expert 3-Tier Routing, Fo-MoE, Skip-Attention & Looped Recurrence

Earlier revisions documented only flat 4-to-8 branch BTM upcycling. Scaling to 64–128 community experts across residential WAN links raises five further design problems. This section describes the components we implemented for them; none has yet been evaluated at the target

scale (the largest MoE actually trained has 4 experts, Section 8.3). A caveat on the router: gating FLOPs are small compared with expert FFN FLOPs, and hierarchical gating restricts which expert combinations are reachable, so the saving below must eventually be weighed against routing quality.

6.1 3-Tier 128-Expert Hierarchical MoE Router (`ThreeTierHierarchicalMoERouter`)

In a flat $E = 128$ Sparse MoE, computing $\mathrm{Softmax}(W_g^\top x)$ requires $E = 128$ inner products of dimension d_{model} per token at every layer, and speculative prefetching over a token tree must evaluate all 128 expert columns. In `src/hivemind/core/btm_moe.py` (`HM-S6-02`), we partition the $E = 128$ experts into a 3-tier hierarchy of $S = 4$ super-clusters $\times M = 4$ macro-domains $\times U = 8$ micro-experts ($4 \times 4 \times 8 = 128$):

$$s^* = \arg\max_{s \in \{1..S\}} P_{\text{super}}(s \mid x), \qquad m^* = \arg\max_{m \in \{1..M\}} P_{\text{macro}}(m \mid x, s^*),$$
$$\mathcal{T}(x) = \text{Top-}k\Big(W_{\text{micro}}^{(s^*,m^*)} x\Big). \tag{5}$$

Per token, the router evaluates only $S + M + U = 4 + 4 + 8 = \mathbf{16}$ inner products of length d_{model} instead of 128—an exact **8.0× gating FLOP reduction** (an operation count; checked in H3 at toy scale). Furthermore, `predict_candidate_experts()` exposes the exact factored path probability $P(e \mid x) = P_{\text{super}}(s^*) P_{\text{macro}}(m^* \mid s^*) P_{\text{micro}}(e \mid s^*, m^*)$ ($\sum_e P(e \mid x) \le 1$) to drive token-tree lookahead DMA prefetching (`prefetch_token_tree_lookahead`).

6.2 Federated Partial Expert Replication (`FoMoE`) & Ghost-Expert `Skip-Token`

Following Sani *et al.* (`FoMoE`, `arXiv:2606.19025`) [10] and Google DeepMind's *Scaling Laws for DiLoCo* (`arXiv:2503.09799`) [6], requiring every consumer GPU to replicate and synchronize all $E = 128$ experts wastes both VRAM and outer-loop WAN bandwidth. In `FoMoEPartialReplicaPlanner` (`src/hivemind/swarm/diloco_nextgen.py`) and `UpcycledSparseMoEModel.set_local_expert_subset()` (`src/hivemind/core/btm_upcycle.py`, `HM-S8-06`), each of the M workers hosts only $M_{\text{loc}} = \frac{O_e}{M} E$ resident experts (where $O_e \in \{1, \dots, M\}$ is the replication overlap factor) while the remaining $E - M_{\text{loc}}$ experts act as **Ghost Experts**:

$$R(O_e) = \frac{2(O_e - 1)}{O_e}, \qquad V_{\text{comm}}(O_e) = \frac{2(M-1)}{M} B_{\text{dense}} + \left(R(O_e) + \gamma_{\text{shuf}} \left(1 - \frac{O_e}{M} \right) \right) M_{\text{loc}} B_{\text{expert}}. \tag{6}$$

During local inner-loop forward passes, any token slot routed by the global router to a non-resident Ghost Expert $e \notin \mathcal{E}_{\text{loc}}$ is dropped locally via **`Skip-Token`**:

$$y_{\text{FoMoE}}(x) = \sum_{e \in \mathcal{T}(x) \cap \mathcal{E}_{\text{loc}}} \tilde{g}_e(x)\, E_{\ell,e}(x), \qquad \text{RPC}_{\text{remote}}(x) = 0. \tag{7}$$

Evaluating this cost model (check H3) with $M = 16$ workers with overlap $O_e = 4$ (32 resident experts and 96 Ghost Experts per worker) predicts a 3.46× reduction in outer ring volume, with no synchronous remote FFN calls because ghost-routed tokens are skipped. This is a model prediction, not a network measurement; the effect of skipping on quality is studied in [10] but not yet in our setting.

σ-MoE routing and measured skip fraction. FoMoE's experts are selected by independent sigmoid gates (σ-MoE, following Csordás *et al.*) rather than a competitive softmax, and its cost model (their Eq. 2–3) uses the ideal scaling $\sigma_{\text{st}} = N_{\text{le}}/N_e$ while explicitly recommending the *measured* ghost-assignment frequency whenever routing is imbalanced.

`FoMoEPartialReplicaPlanner` models only the ideal factor; `src/hivemind/core/fomoe_sigma_routing.py` adds `sigmoid_independent_topk_routing` (global router, ghost slots skipped *after* routing, realised ghost fraction returned per batch) and `skip_token_flops_per_token` (their Eq. 3 with either factor, and the N_{le}/N_e gate factor only when the router itself is partitioned). On the toy two-layer 128-expert configuration of `tests/test_fomoe_sigma_routing.py` (32 resident experts, top-2) the ideal factor and a measured 50% ghost fraction give 1,432 vs. 1,832 FLOPs per token, so the ideal factor under-estimates compute by 28% under that imbalance. Neither function has been run on a trained model.

6.3 `Skip-Attention` Speculative Prefetch & Cache-Aware Routing (`CAR`)

When serving all E experts locally via 3-tier NVMe → Pinned Host RAM → VRAM streaming, waiting until Layer $\ell+1$'s self-attention finishes to evaluate Layer $\ell+1$'s router exposes the PCIe DMA transfer latency. Inspired by the multi-tier expert caching and cache-aware routing of Paul Merolla's `fomoe` inference engine [11] and by expert prefetching in `learning-at-home/hivemind` [27], our `SkipAttentionSpeculativeRouter` (`src/hivemind/swarm/diloco_nextgen.py`, `HM-S8-07`) evaluates Layer $\ell+1$'s RMSNorm and router projection $W_{g,\ell+1}$ directly on Layer ℓ's **post-FFN residual stream** $x_\ell^{\text{post-FFN}}$ *before* Layer $\ell+1$'s self-attention begins:

$$\hat{\mathcal{T}}_{\ell+1}^{\text{spec}}(x) = \text{Top-}k\Big(W_{g,\ell+1}\,\text{RMSNorm}_{\ell+1}\Big(x_\ell^{\text{post-FFN}}\Big)\Big). \tag{8}$$

The predicted experts $\hat{\mathcal{T}}_{\ell+1}^{\text{spec}}$ are streamed asynchronously from Pinned Host RAM into the alternating lookahead CUDA double-buffer slot (`slot_A`/`slot_B`) in parallel with Layer $\ell+1$'s self-attention compute. Once self-attention finishes and true post-attention router logits $z^{\text{true}} \in \mathbb{R}^E$ are available, any selected expert $e^* \notin \mathcal{C}_{\text{VRAM}}$ that missed the speculative prefetch is checked against the **Cache-Aware Routing (`CAR`)** threshold τ_{CAR}:

$$\text{If } \min_{e_c \in \mathcal{C}_{\text{VRAM}} \setminus \mathcal{T}} \Big(z_{e^*}^{\text{true}} - z_{e_c}^{\text{true}}\Big) \le \tau_{\text{CAR}}, \quad \text{substitute } e^* \leftarrow \arg\max_{e_c \in \mathcal{C}_{\text{VRAM}} \setminus \mathcal{T}} z_{e_c}^{\text{true}}. \tag{9}$$

In unit check H5 the speculative Top-2 prediction matched the true routing in 95.83% of cases. That figure is *not* a property of any trained model: the check uses random 32-dimensional states and sets the post-attention state to the post-FFN state plus Gaussian noise of scale 0.08, so the accuracy is determined by that noise level. Measuring speculative accuracy on real Qwen2.5 activations is future work.

6.4 Weight-Tied `LoopedExpertBackbone`, Fixed-Point Halting & Truncated BPTT

To multiply reasoning depth on 8 GB consumer GPUs without multiplying model weight VRAM, `LoopedExpertBackbone` (`src/hivemind/core/btm_upcycle.py`, `HM-S7-01/02`) wraps an upcycled Sparse MoE decoder stack into a three-stage recurrent architecture synthesized from *Towards Looped Models Done Right, Part I: Topology, Input Injection, Recurrent-State Design* (`arXiv:2610.23036`) [12] and *Part II: Rethinking at Fixed Points* (`arXiv:2610.06833`) [13]:

$$\begin{aligned} h^{(0)} &= \text{Prelude}(x), \qquad y = \text{Coda}\Big(h^{(T)}\Big), \\ h^{(t)} &= \text{RecurrentCore}\Big(\mathcal{A}_t\Big(h^{(t-1)} + \mathbb{I}[t>1]h^{(0)}\Big)\Big) \quad (t = 1,\dots,T), \end{aligned} \tag{10}$$

where $\mathcal{A}_t(u) = u + B_t^{\text{step}} A_t^{\text{step}} u$ is a lightweight per-step residual LoRA adapter initialized with $B_t^{\text{step}} = 0$ (exact identity at initialization). `LoopedExpertBackbone` implements three systems optimizations (the associated Lean lemmas only count bytes):

1. **Zero Duplicate Core Weight VRAM:** The recurrent core layers are weight-tied across all T loop steps, saving $(T-1)B_{\text{core}}$ bytes of weight storage (669,888 B saved at $T=4$, multiplying effective depth from $4 \to 10$ layers with only 4,096 B of step adapters).

2. **Per-Token Fixed-Point Convergence Early Exit:** After each recurrent pass t, each active token position i evaluates its relative state drift $r_i^{(t)} = \|h_i^{(t)} - h_i^{(t-1)}\|_2 / \max(\|h_i^{(t-1)}\|_2, 10^{-6})$. Tokens satisfying $r_i^{(t)} \leq \epsilon_{\text{fp}}$ freeze their hidden state for subsequent passes while remaining visible as causal attention context.

3. **RNG-Restored Truncated Fixed-Point BPTT (`truncated_bptt_steps`):** When $k_{\text{bptt}} < T$ and adaptive halting is active, forward passes discover the exact per-token halting trajectory without storing intermediate activation graphs, then replay only the trailing k_{bptt} recurrent passes under `torch.random.fork_rng` with restored CPU/CUDA RNG state and preserved MoE routing counters.

   In unit check H4 (`fixed_point_tol = 0.65` with $T=4$ loop steps), tokens halt after a non-trivial mean of **3.05** recurrent passes (80% early-exit rate, mean residual 0.5215). In Experiment E3 (Section 8, Figure 3a), sweeping $\epsilon_{\text{fp}} \in [0.0, 1.10]$ across a $T=6$ looped MoE backbone maps the smooth trade-off between recurrent FLOP savings ($0\% \to 71.88\%$) and relative output logit drift ($0\% \to 58.61\%$).

6.5 Subspace-Aware Attention & Expert Merging: `Iso-CTS` & `TSV`

Standard BTM upcycling uniformly averages self-attention LoRA deltas ($\bar{\Delta}_{\text{attn}} = \frac{1}{E}\sum_{e=1}^{E} \Delta_{\text{attn},e}$), which suffers from singular-direction dominance collapse when heterogeneous domain branches are combined. Building on isotropic merging (Iso-C / Iso-CTS) by Marczak *et al.* [15], its use as a DiLoCo aggregator in IsoLoCo by Horoi *et al.* [16], and Task Singular Vectors by Gargiulo *et al.* [17], `upcycle_branches_to_sparse_moe()` supports two spectral subspace mergers (`HM-S8-03`, **Hypothesis H6**):

- **Reputation-Weighted `ISO-CTS` (`attention_merge_mode="iso_cts"`):** Elects sign consensus via reputation-weighted TIES, extracts the top $c = \lfloor \rho_c \min(d_{\text{out}}, d_{\text{in}}) \rfloor$ common singular vectors U_{common}, projects worker residuals onto the orthogonal complement $(I - U_{\text{common}}U_{\text{common}}^\top)$, orthogonalizes the combined basis via SVD Procrustes projection, and flattens the singular value spectrum to its isotropic mean $\bar{\sigma} = \frac{1}{r}\sum_i \sigma_i$, which makes the merged singular-value condition number exactly 1 by construction (H6 confirms $2.5988 \to 1.0000$ on a toy merge; whether this helps downstream quality in our setting is untested).

- **Task Singular Vector Decorrelation (`attention_merge_mode="tsv"`):** Extracts the top $k_w = \lfloor d_{\min}/M \rfloor$ singular triplets $(U_m, \Sigma_m, V_m^\top)$ per branch and whitens inter-branch subspace interference via orthogonal Procrustes projection $U_{\text{orth}} = P_u Q_u^\top, V_{\text{orth}}^\top = P_v Q_v^\top$, achieving $\|U_{\text{orth}}^\top U_{\text{orth}} - I\|_F = \mathbf{6 \times 10^{-6}}$ (**Theorem 110**).

7 OpenDiLoCo & Google DeepMind Outer-Loop Ecosystem

To make outer-loop aggregation resilient to asynchronous WAN stragglers, residential upload limits, and 8 GB consumer GPU memory ceilings, Project HiveMind adapts ideas and, where licences allow, code from eight open-source repositories and recent papers from Google DeepMind, Prime Intellect, Mila, Covenant AI, Nous Research, and Cambridge / Flower Labs (Table 2). We credit these works for the underlying methods; HiveMind's contribution is their integration under an 8 GB memory budget.

Table 2: Open-Source GitHub Repositories & Foundational Papers Integrated into Project HiveMind's `OpenDiLoCo` + `Sparse MoE` Architecture (`src/hivemind/swarm/diloco_nextgen.py`).

Repository / Project	Primary Authors & Lab	Reference	Integrated HiveMind Module & Role
`google-deepmind/asyncdiloco`	Liu, Chhaparia, Douillard *et al.* (DeepMind)	[3, 4]	Reference for asynchronous outer updates; our fragment streaming (`synced_fragments`) follows Streaming DiLoCo
`Decoupled DiLoCo` (paper only; no public code)	Douillard, Rush, Donchev, Charles *et al.* (DeepMind & Google Research)	[5]	`DecoupledDiLoCoQuorumSyncer` (`decoupled_diloco.py`): minimum quorum K, adaptive grace window, token weights, RDA merge
`PrimeIntellect-ai/OpenDiLoCo`	Jaghouar, Ong, Hagemann (Prime Intellect)	[7]	Decentralized outer-step orchestration & Hivemind P2P fault-tolerant gradient averaging
`PrimeIntellect-ai/prime-diloco`	Prime Intellect Team	[7]	`CPUOffloadedFlatBucketOuterState`: contiguous 1D FP32 CPU master buffer (`0 B` extra GPU VRAM)
`one-covenant/SparseLoCo`	Sarfi, Belilovsky *et al.* (Mila / Covenant)	[19, 20]	`ComputeVisibleSparseLoCoCompressor` & `DCTSparse2BitTensorPayload` (2-bit 4-bin LUT + EF)
`bloc97/DeMo`	Peng, Quesnelle, Kingma (Nous Research)	[18]	`ChunkedOrthonormalDCTTransform`: 8×8 Type-II orthonormal DCT with deterministic `index_copy_`
`shoroi/isoloco`	Horoi, Thérien, Wolf, Belilovsky (Mila)	[16, 15]	`ReputationWeightedIsoCTSMerger` (`ISO-C`/`ISO-CTS`) & `TSVDecorrelatedMerger`
`pmerolla/fomoe`	Paul Merolla	[11]	Multi-tier expert cache & cache-aware routing; inspires `SkipAttentionSpeculativeRouter` (FoMoE partial replication follows [10])
`learning-at-home/hivemind`	Ryabinin, Diskin, Mazur *et al.*	[27, 28]	Kademlia DHT peer rings, background expert prefetching & decentralized MoE routing

7.1 `OpenDiLoCo` & `prime-diloco`: Contiguous 1D CPU Flat-Bucket Outer State (`0 B` GPU VRAM)

In naive PyTorch DiLoCo implementations, holding a copy of the outer master weights $\theta^{(t-1)}$ and outer Nesterov velocity buffer $v^{(t)}$ on `cuda:0` doubles the adapter/backbone memory footprint during every outer step. Adapting `PrimeIntellect-ai/prime-diloco` (`src/zeroband/diloco.py`) and `PrimeIntellect-ai/OpenDiLoCo` (`arXiv:2407.07852`) [7], `CPUOffloadedFlatBucketOuterState` (`src/hivemind/swarm/diloco_nextgen.py`) and `DiLoCoOuterOptimizer(use_cpu_flat_bucket=True)` (`src/hivemind/core/diloco_outer.py`, `HM-S8-02`) pack all parameter fragments into a single contiguous 1D FP32 host RAM allocation $\mathbf{w}_{\text{flat}} \in \mathbb{R}^N_{\text{CPU}}$ with deterministic slice offsets $\{[\text{lo}_k, \text{hi}_k)\}_{k=1}^F$:

$$\begin{aligned} \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] &\leftarrow \mu \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] + \bar{\Delta}_k^{(t)}, \\ \mathbf{w}_{\text{flat}}[\text{lo}_k : \text{hi}_k] &\leftarrow \mathbf{w}_{\text{flat}}[\text{lo}_k : \text{hi}_k] - \eta \left(\mu \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] + \bar{\Delta}_k^{(t)} \right). \end{aligned} \tag{11}$$

By construction the outer state then uses no GPU memory; the cost moves to host RAM and to one host-to-device copy per outer step, which we have not yet timed. Check H1 confirms the flat-bucket update matches the per-tensor Nesterov update to single-ULP FP32 precision ($\max \|\Delta\theta\|_\infty = 2.384 \times 10^{-7}$).

7.2 Decoupled DiLoCo: Minimum Quorum, Adaptive Grace Window & Radial-Directional Averaging

Google DeepMind's *Decoupled DiLoCo* (`arXiv:2604.21428`) [5] removes the lock-step outer all-reduce altogether: M independent learners report to a CPU-resident *syncer* that aggregates a fragment as soon as a minimum quorum of $K \leq M$ learners has arrived, optionally waits an adaptive grace window for late learners, weights each learner by its token throughput, and merges the per-learner outer gradients with Radial-Directional Averaging (RDA). This is the regime a volunteer swarm actually lives in (stragglers, churn, heterogeneous GPUs), and until this revision HiveMind cited the paper but implemented none of it. `DecoupledDiLoCoQuorumSyncer` (`src/hivemind/swarm/decoupled_diloco.py`, `SI-12`) implements the four mechanisms of Algorithm 2 of [5] as separately testable pieces:

1. **Minimum quorum.** The first K learners by arrival are always admitted; a round with fewer than K reports does not merge (**Theorem 120** fixes $K \le |\mathcal{A}| \le M$ and shows that the degenerate $K = 0$ rule admits nobody).

2. **Adaptive grace window.** With $\xi_{\text{slack}} = \tau\,\xi_{\text{step}} - (\xi_{\text{quorum}} + \xi_{\text{sync}})$ (their Eq. 3) tracked by exponential moving averages, learners arriving within $\xi_{\text{grace}} = \gamma \max(0, \xi_{\text{slack}})$ of the quorum instant are also admitted. **Theorems 117–118** check that $\gamma < 1$ keeps $\xi_{\text{grace}} < \xi_{\text{slack}}$ whenever the slack is positive and that an exhausted budget gives $\xi_{\text{grace}} = 0$.

3. **Token-weighted merging.** $w_m = c_m^{\text{tokens}} \cdot c_m^{\text{tokens}} / c_m^{\text{steps}}$ ("quantity × quality"; a learner with no inner step has weight 0; **Theorem 119**).

4. **Radial-Directional Averaging.** $\text{RDA}(\{(\Delta_m, w_m)\}) = \big(\sum_m \bar{w}_m \|\Delta_m\|\big)\, \phi\big(\sum_m \bar{w}_m \phi(\Delta_m)\big)$ with $\phi(x) = x/\|x\|$ and $\bar{w}_m = w_m / \sum_j w_j$, so that equal-norm, nearly orthogonal learner updates keep their norm R instead of shrinking to $R/\sqrt{M}$ under direct averaging; the outer learning rate therefore no longer needs retuning with M.

The syncer emits Δ_p and leaves the outer optimizer untouched: check `H11` chains it into the CPU flat-bucket Nesterov step above with `0 B` extra GPU memory. In `H11` (`artifacts/decoupled_diloco_quorum_check.json`; three arms, one positive and two negative controls) RDA returns norm R for $M \in \{2, 4, 8, 16\}$ orthogonal equal-norm deltas while the direct mean follows $R/\sqrt{M}$ exactly (RDA-to-direct ratio **2.8284** at $M = 8$); with $\tau = 2$, $\xi_{\text{step}} = 4.0\,\text{s}$, $\xi_{\text{quorum}} = 0.3\,\text{s}$, $\xi_{\text{sync}} = 1.0\,\text{s}$ and $\gamma = 0.5$ the controller opens a **3.35** s window on 6.7 s of slack, admits 6/8 learners, defers two stragglers carrying 10,000 of 40,000 tokens, and the stragglers' coordinates do not leak into Δ_p (leak = 0.0); a $K = M = 8$ quorum with seven reports correctly refuses to merge. These are bookkeeping checks on synthetic tensors: the goodput and quality results under chaos engineering are DeepMind's, not ours, and nothing here has yet run across machines.

7.3 Streaming DiLoCo Fragments & Cosine-Gated Adam-Decay (`PA-CGAD`)

Google DeepMind's `Streaming DiLoCo` (`arXiv:2501.18512`) [4] reduces peak WAN bandwidth by synchronizing only a subset of parameter fragments $\mathcal{S}_t \subset \{1, \dots, F\}$ per outer round (**Theorem 102**). However, under asynchronous WAN delays τ_w and staggered fragment ages a_f, standard outer Adam/Nesterov suffers from *stale variance pollution*: stale pseudo-gradients inflate the second-moment buffer v_f, shrinking future honest step sizes. Following V. Shah and J. Sun (`CGAD / PA-CGAD`, `arXiv:2605.09126`) [14], `CosineGatedAdamDecayOuterOptimizer` (`HM-S8-01`) defines the effective fragment staleness $\tau_{\text{eff}}(f) = \max(0, \tau_w(f), a_f)$ and gates $\bar{\Delta}_f^{(t)}$ **before** updating (m_f, v_f):

$$\gamma(\tau) = \frac{1}{2}\left(1 + \cos\left(\frac{\pi\tau}{\tau_{\text{cut}}}\right)\right) \mathbb{I}[\tau < \tau_{\text{cut}}], \qquad \sigma(\tau) = \gamma(\tau) e^{-\alpha\tau}, \qquad \tilde{g}_f = \sigma(\tau_{\text{eff}}(f))\, \bar{\Delta}_f^{(t)}. \tag{12}$$

When $\tau_{\text{eff}}(f) \ge \tau_{\text{cut}}$, $\sigma(\tau_{\text{eff}}(f)) = 0$ and the fragment's moment buffers (m_f, v_f) are completely frozen (**Theorems 78, 103**). Furthermore, for all $\tau \ge 0$, the staleness bias satisfies the uniform upper bound:

$$\sup_{\tau \ge 0} \tau\sigma(\tau) \le \sup_{\tau \ge 0} \tau e^{-\alpha\tau} = \frac{1}{e\alpha}. \tag{13}$$

This bound is analytic and is due to [14]. Check H2 only confirms that our implementation of σ respects it ($\max_{\tau \in [0,64]} \tau\sigma(\tau) = 1.7417 \le 1.8394$ at $\alpha = 0.2$, $\tau_{\text{cut}} = 32$) and that cut-off fragments leave (m_f, v_f) untouched; it says nothing about training dynamics under real delays.

7.4 `DeMo` Chunked 2D DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo` + `EF21`

To compress outer pseudo-gradients far beyond 8-bit `float8_e4m3fn`, `src/hivemind/core/compression.py` (`compression_mode="dct_sparse_2bit"`, `HM-S8-04/05`) unifies three complementary algorithms:

1. **Compute-visible gate after `PULSELoCo` (`arXiv:2602.03839`, introduced for distributed RL post-training) [20]:** Drops per-coordinate updates $|\Delta_i| < 2^{-8}|\theta_i|$ that would round to zero when added to a BF16/FP16 parameter (**Theorem 81**).

2. **`DeMo` Orthonormal 2D Chunked Type-II DCT (`bloc97/DeMo`, `arXiv:2411.19870`) [18]:** Reshapes each LoRA delta into 8×8 blocks and projects them via the orthonormal Type-II DCT basis $C \in \mathbb{R}^{8\times8}$ ($CC^\top = I$, measured $\|CC^\top - I\|_F = 2.3 \times 10^{-7}$), concentrating spatial gradient energy into low-frequency coefficients.

3. **`SparseLoCo` 2-Bit Statistical Centroid LUT & Bit-Packing (`one-covenant/SparseLoCo`, `arXiv:2508.15706`) [19]:** Retains the top 2% DCT frequency coefficients, quantizes them into 4 empirical statistical centroids stored as a 4-entry FP16 lookup table (8 bytes), and packs four 2-bit codes per `uint8` byte (`_pack_2bit_codes`) with deterministic `index_copy_` reconstruction (eliminating `DeMo`'s non-deterministic CUDA `scatter_reduce_`).

In check H7 a package of four 256×16 LoRA deltas is encoded in 820 B. Measured against 16-bit storage (32,768 B) this is 39.96×; against FP32 (65,536 B) it is 79.92×. Because a single 2% Top-k projection on isotropic Gaussian noise has a high single-shot relative L_2 truncation error ($\|\hat{\Delta} - \Delta\|_2 / \|\Delta\|_2 = 0.9313$ for 2-bit DCT and 0.8966 for PULSE+SparseLoCo), pairing aggressive 2-bit sparsification with **EF21 error-feedback residual accumulation** [26] is essential: `EF21DiLoCoErrorFeedbackBuffer` (**Theorems 57, 107**, **Hypothesis H10**) preserves exact multi-step telescoping sum conservation $\|(\sum_{t=1}^{3} c_t + r_3) - \sum_{t=1}^{3} g_t\|_2 = \mathbf{4 \times 10^{-8}}$, and Experiment E1 (Section 8, Figure 1) demonstrates that 2-bit DCT + EF21 matches uncompressed FP32 DiLoCo within **0.052%** held-out validation loss across 12 outer rounds.

7.5 `AutoLoCo`, `HeLoCo` & Inner-Loop `MuLoCo` Newton-Schulz Preconditioning

Finally, to stabilize heterogeneous worker cadences and large replica counts:

- **`AutoLoCo` (`arXiv:2609.36662`) [22] & `HeLoCo` (`arXiv:2606.00271`) [21]:** When `AdaptiveDiLoCoCadenceController` adapts inner step count H_t, `AutoLoCoIntervalCorrector` scales outer learning rate $\eta_{\text{outer}} \leftarrow \eta_0 / \sqrt{S_t / S_{\text{ref}}}$ and outer momentum $\mu_{\text{outer}} \leftarrow \mu_0^{S_t/S_{\text{ref}}}$ (0.7000 → 0.4950 when $H : 32 \to 64$, **Theorem 84**), while `HeLoCoDirectionCorrector` projects out conflicting stale drift components whenever $\langle g_k, m_{\text{ref}} \rangle < 0$ (**Theorem 85**, **Hypothesis H8**).

- **Muon-style quintic Newton-Schulz LoRA preconditioning (Muon [23]; MuLoCo, `arXiv:2505.23725` [24]; replica-scaling analysis, `arXiv:2610.19064` [25]):** In `run_diloco_inner_loop(..., use_muon_lora_preconditioner=True)` (`src/hivemind/core/diloco_inner.py`, `HM-S8-09`), 2D LoRA gradients $G \in \mathbb{R}^{m\times n}$ are normalized ($X_0 = G/\|G\|_F$) and orthogonalized via $K = 5$ quintic Newton-Schulz iterations with the coefficients of Muon [23] ($a = 3.4445, b = -4.7750, c = 2.0315$):

$$X_{k+1} = aX_k + \left(bX_kX_k^\top + c\left(X_kX_k^\top\right)^2\right)X_k. \tag{14}$$

  In check H9, 5 iterations reduce the condition number of a synthetic 32×16 rank-8 gradient from 125.0 to 1.65, and validator replay on CPU reproduces the worker bit-for-bit (difference 0.0). Cross-GPU bitwise reproducibility was not tested.

7.6 Analytical Theorems, Proofs, Hypothesis-Necessity Counterexamples & 4-Definitions Contract

To make the mathematical guarantees and their failure modes self-contained, we state the three core outer-loop invariants with explicit analytical proofs and paired *Hypothesis-Necessity Counterexamples* (`Season13`, Lean 4 Theorems 113–116), followed by the 4-Definitions Contract (Table 3).

Theorem 7.1 (Multi-Step EF21 Telescoping Conservation & Necessity of Residual Carry; Lean Thms. 107 & 113)**.** *Let $\{g_t\}_{t=1}^T \subset \mathbb{R}^d$ be a sequence of outer pseudo-gradients and $\mathcal{C} : \mathbb{R}^d \to \mathbb{R}^d$ be any deterministic compressor (including 2-bit `DeMo` DCT + `SparseLoCo` LUT). Under the error-feedback recurrence $u_t = g_t + r_{t-1}$, $c_t = \mathcal{C}(u_t)$, $r_t = u_t - c_t$ with $r_0 = 0$, the cumulative transmitted sum satisfies the exact telescoping identity:*

$$\sum_{t=1}^{T} c_t + r_T = \sum_{t=1}^{T} g_t. \tag{15}$$

Hypothesis-Necessity Counterexample (Thm. 113): *If error feedback is disabled ($r_t \equiv 0$, so $c_t = \mathcal{C}(g_t)$) and step 1 compression is strictly contractive on positive mass ($c_1 < g_1$ with $c_2 \le g_2$), then $c_1 + c_2 < g_1 + g_2$ with positive permanent deficit $g_1 - c_1 > 0$.*

Proof. At each round $t \in \{1, \dots, T\}$, substituting $u_t = g_t + r_{t-1}$ into $c_t + r_t = u_t$ yields $c_t = g_t + r_{t-1} - r_t$. Summing from $t = 1$ to T telescopes all intermediate residuals $r_1, \dots, r_{T-1}$:

$$\sum_{t=1}^{T} c_t = \sum_{t=1}^{T} g_t + r_0 - r_T = \sum_{t=1}^{T} g_t - r_T \implies \sum_{t=1}^{T} c_t + r_T = \sum_{t=1}^{T} g_t.$$

Conversely, when $r_1 = 0$, step 2 compresses g_2 alone ($c_2 \le g_2$), so $c_1 < g_1 \implies c_1 + c_2 < g_1 + g_2$ (machine-checked in `ef21_disabled_loses_untransmitted_residual`). This explains why single-round 2-bit DCT exhibits 0.9313 relative L_2 truncation error in `H7` yet achieves a 0.052% held-out loss gap across $R = 12$ rounds with EF21 in Experiment `E1`. □

Theorem 7.2 (PA-CGAD Staleness Bias Bound & Necessity of Cosine Cutoff; Lean Thms. 103 & 114)**.** *For any decay rate $\alpha > 0$ and cutoff horizon $\tau_{cut} > 0$, the PA-CGAD gate $\sigma(\tau) = \frac{1}{2}(1 + \cos(\pi\tau/\tau_{cut}))\,\mathbb{I}[\tau < \tau_{cut}]e^{-\alpha\tau}$ satisfies:*

$$\sup_{\tau \ge 0} \tau\sigma(\tau) \le \frac{1}{e\alpha}, \qquad \text{and} \qquad \forall \tau_{eff} \ge \tau_{cut}, \quad v_t = \beta_2 v_{t-1}. \tag{16}$$

Hypothesis-Necessity Counterexample (Thm. 114): *If the staleness cutoff is omitted ($\alpha_2 = 1$ at $\tau_{eff} \ge \tau_{cut}$), any non-zero stale gradient $\|g_{stale}\|_2^2 > 0$ with $1 - \beta_2 > 0$ strictly inflates the second moment: $v_{unfiltered} - v_{clean} = (1 - \beta_2)\|g_{stale}\|_2^2 > 0$.*

Proof. Since $0 \le \gamma(\tau) \le 1$ for all $\tau \ge 0$, we have $0 \le \tau\sigma(\tau) \le \phi(\tau) := \tau e^{-\alpha\tau}$. Differentiating $\phi(\tau)$ gives $\phi'(\tau) = (1 - \alpha\tau)e^{-\alpha\tau}$, which vanishes uniquely at $\tau^* = 1/\alpha$ with $\phi''(1/\alpha) = -\alpha e^{-1} < 0$. Hence $\sup_{\tau \ge 0} \tau\sigma(\tau) \le \phi(1/\alpha) = \frac{1}{e\alpha}$ (at $\alpha = 0.2$, $\frac{1}{0.2e} \approx 1.8394$, bounding the empirical maximum 1.7417). For $\tau_{\text{eff}} \ge \tau_{\text{cut}}$, $\mathbb{I}[\tau_{\text{eff}} < \tau_{\text{cut}}] = 0 \implies \tilde{g}_f = 0$, preserving v_t without stale variance inflation. When omitted ($\alpha_2 = 1$), $v_{\text{unfiltered}} = \beta_2 v_{t-1} + (1 - \beta_2) g_{\text{stale}}^2 > \beta_2 v_{t-1} = v_{\text{clean}}$ (machine-checked in `pacgad_missing_cutoff_inflates_stale_second_moment`, explaining the 11.6× MSE degradation in Experiment `E2`). □

Theorem 7.3 (Multi-Krum Honest Quorum & Violation Counterexample; Lean Thms. 4 & 115)**.** *In a swarm of n workers with at most f Byzantine adversaries, if $n \ge 2f + 3$, then the Multi-Krum neighborhood size $k = n - f - 2$ satisfies $k \ge f + 1$, so every k-nearest neighborhood contains at least one honest worker.* ***Hypothesis-Necessity Counterexample (Thm. 115):*** *If $n \le 2f + 2$, then $k = n - f - 2 \le f$, so f colluding Byzantine workers can fill the entire k-neighborhood (`multikrum_violated_quorum_loses_honest_majority`).*

Proof. Subtracting $f+2$ from $n \geq 2f+3$ yields $k = n-f-2 \geq (2f+3)-f-2 = f+1$. By the pigeonhole principle, since at most f workers are Byzantine, at least $k-f \geq 1$ selected neighbors are honest. Conversely, if $n \leq 2f+2$, then $k = n-f-2 \leq f$. □

Table 3: 4-Definitions Contract mapping each core subsystem across its Mathematical Operator (Def. A), Conservation/Lyapunov Invariant (Def. B), Discretization/Algorithm (Def. C), and Quantitative Verification Gate (Def. D).

Subsystem	Def. A: Operator	Def. B: Invariant	Def. C: Algorithm	Def. D: Quantitative Gate
2-Bit DCT + EF21	$\hat{\Delta}_t = \mathcal{C}_{2b}(g_t + r_{t-1})$	$\sum_{t=1}^{T} \hat{\Delta}_t + r_T = \sum_{t=1}^{T} g_t$	8 × 8 DCT-II + 4-bin LUT + `index_copy_`	39.96× wire savings; 0.052% loss gap (`E1`, Thms. 105, 107, 113)
PA-CGAD Outer	$\tilde{g}_f = \gamma(\tau_{\text{eff}})e^{-\alpha\tau_{\text{eff}}}\bar{\Delta}_f$	$\sup_{\tau\geq 0} \tau\sigma(\tau) \leq \frac{1}{e\alpha}$	Pre-AdamW staleness cutoff at $\tau_{\text{cut}} = 32$	1.7417 ≤ 1.8394; 11.6× lower straggler MSE (`E2`, Thms. 103, 114)
MuLoCo NS-5	$\Phi(X) = aX + (bXX^\top + c(XX^\top)^2)X$	$\sigma_i(\Phi^{(5)}(X_0)) \to 1$	5-step quintic Newton-Schulz on 2D LoRA grads	$\kappa : 125.0 \to 1.65$; replay diff = 0.0 (`H9`, `E1`, Thm. 106)
Looped MoE	$h^{(t)} = \text{Core}(\mathcal{A}_t(h^{(t-1)} + h^{(0)}))$	$r_i^{(t)} \leq \epsilon_{\text{fp}} \implies h_i^{(t+1)} = h_i^{(t)}$	Weight-tied core + RNG-restored TBPTT	669,888 B saved; $\bar{t} = 3.05/4.0$ at $\epsilon_{\text{fp}} = 0.65$ (`H4`, `E3`, Thm. 108)
Multi-Krum + Spot	$\mathcal{N}_k(i) = \text{KNN}_{n-f-2}(\Delta_i)$	$n \geq 2f+3 \implies \lvert\mathcal{N}_k \cap \mathcal{H}\rvert \geq 1$	10% SHA-256 micro-segment replay + 2σ cosine	20× BOINC FLOP savings; 25/25 red-team vectors blocked (Thms. 4, 115)

8 Empirical Verification, Controlled Experiments & Visualizations

8.1 Unit-Level Implementation Checks (`H1`–`H10`)

Each check in `src/hivemind/swarm/architecture_hypothesis_verifier.py` (`pytest tests/test_architecture_hypotheses.py`) exercises one architectural subsystem in isolation on CPU with controlled tensors (hidden size 32–64, LoRA rank 4–16, up to 128 router outputs) and emits the cryptographically signed report `artifacts/architecture_hypothesis_verification_report.json` (`receipt_sha256: b427d910a2d3ba4c...`). Table 4 summarizes all 10 checks.

Check H11 (Decoupled DiLoCo quorum syncer). `scripts/verify_decoupled_diloco_quorum.py` writes `artifacts/decoupled_diloco_quorum_check.json` with one positive arm (RDA norm invariance vs. direct-average shrinkage for $M \in \{2, 4, 8, 16\}$) and two negative controls (late stragglers excluded by the grace window; a $K = M$ quorum with one missing learner refuses to merge). All three arms pass; the numbers are quoted in Section 7.2, and the provenance lock (`PROV-15`, `PROV-16`) ties the 2.8284 ratio and the 3.35 s window in the text to that artifact.

8.2 Controlled Experimental Evaluations (`E1`–`E4`) & Visualizations

To move beyond isolated unit checks, `scripts/run_paper_experiments_and_figures.py` executes four controlled experimental studies (`E1`–`E4`) and records all curves and SHA-256 provenance digests in `artifacts/paper_v3_experiments_report.json` (`receipt_sha256: 8cba94a46af02f8a...`).

Table 4: Unit-level implementation checks (`artifacts/architecture_hypothesis_verification_report.json`, `receipt_sha256: b427d910a2d3ba4c...`). ‡H5's speculative Top-2 match rate is measured on synthetic Gaussian residual perturbations ($\sigma = 0.08$).

ID	Architectural Hypothesis	Measured Empirical Telemetry	Lean lemma	Check
H1	`OpenDiLoCo` / `prime-diloco` 1D CPU Flat-Bucket Outer State	`0 B` extra GPU VRAM; contiguous 5,120 B buffer; $\max \Delta\theta = 2.384 \times 10^{-7}$ vs. Nesterov	Thms. 86, 101	pass
H2	DeepMind `Streaming DiLoCo` + `PA-CGAD` Staleness Bias Bound	$\max_\tau \tau\sigma(\tau) = 1.7417 \le \frac{1}{e\alpha} = 1.8394$; $\sigma(\tau \ge 32) = 0$; v_{stale} unpolluted	Thms. 78, 102, 103	pass
H3	128-Expert 3-Tier Router + `FoMoE` Partial Replica & `Skip-Token`	16 vs. 128 dot products (**8.0**× **FLOP savings**); **3.4615**× WAN ring savings; 51/96 ghost skips	Thms. 42, 79, 80, 109	pass
H4	`LoopedExpertBackbone` Weight-Tied Core, Fixed-Point Exit & TBPTT	Depth 4 → 10 (2.5×); 669,888 B saved; $\epsilon_{\text{fp}} = 0.65 \Rightarrow$ **3.05**/**4.0** mean steps (80% early exit)	Thms. 88, 91, 92, 108	pass
H5‡	`Skip-Attention` Prefetch + `CAR` Fallback + 3-Tier Async DMA	95.83% speculative Top-2 match on synthetic states; DMA overlap simulated (0.0 ms exposed)	Thms. 12, 82, 111	pass
H6	`IsoLoCo` (`ISO-CTS`) Isotropic Spectrum & `TSV` Decorrelation	Condition number $\kappa : 2.5988 \to$ **1.0000**; TSV orthogonality error $\|U^\top U - I\|_F = 6 \times 10^{-6}$	Thms. 83, 97, 110	pass
H7	`DeMo` 2D Chunked DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo`	$\|CC^\top - I\|_F = 2.3 \times 10^{-7}$; 32,768 B → 820 B (**39.96**× vs. FP16); single-shot rel. L_2 = 0.9313	Thms. 81, 87, 105	pass
H8	`AutoLoCo` Interval Scaling & `HeLoCo` Half-Space Drift Projection	$\eta_{\text{outer}} : 0.7000 \to 0.4950$ ($H : 32 \to 64$); conflicting cos : $-0.8083 \to 0.0000$ ($\langle g_{\text{corr}}, m\rangle = 0$)	Thms. 84, 85	pass
H9	`MuLoCo` 5-Step Quintic Newton-Schulz LoRA Preconditioning	Singular condition number $\kappa : 125.0001 \to$ **1.6495**; validator spot-check replay diff = **0.0**	Thms. 90, 106	pass
H10	`EF21-DiLoCo` Telescoping Sum Conservation & `FP8PagedKVCache`	3-round telescoping error = $\mathbf{4 \times 10^{-8}}$; KV page 16,384 B → 8,448 B (**1.9394**× compression)	Thms. 38, 57, 107	pass

8.2.1 Experiment E1: Held-Out Generalization & Wire Efficiency Across Five DiLoCo / MuLoCo Regimes

We instantiate a 2-layer causal Qwen2-QLoRA model ($V = 256$, $d = 48$, $r = 8$, $\alpha = 16$) and generate a synthetic Zipfian-transition corpus partitioned into $K = 4$ disjoint non-IID worker training shards (64 sequences/worker, 256 total training sequences) and a strictly disjoint held-out validation split ($N_{\text{val}} = 128$ sequences). All five regimes start from identical initial weights (initial validation loss 3.71488, perplexity 41.0538) and execute $R = 12$ outer synchronization rounds of $H = 8$ local inner steps each (96 inner updates/worker). Table 5 and Figure 1 report the held-out trajectories and cumulative outer wire volumes:

- **2-Bit DCT + EF21 Wire Compression Gate (`PASS`):** `DiLoCo + DeMo 2D-DCT + SparseLoCo 2-Bit + EF21` achieves a final held-out loss of **3.70814** (PPL = 40.7779) vs. **3.70622** (PPL = 40.6996) for uncompressed FP32 DiLoCo—a relative validation loss gap of just **0.052%** (well within the pre-declared 2.0% gate), while reducing cumulative outer wire traffic from 384.0 KB to **30.75** KB (**12.49**× reduction vs. FP32 DiLoCo, and **99.9**× reduction vs. synchronous DDP's 3,072.0 KB).
- **Inner-Loop `MuLoCo` Newton-Schulz Preconditioning:** Adding 5-step quintic Newton-Schulz LoRA gradient orthogonalization inside the compressed 2-bit EF21 regime (`muloco_ns5_dct_2bit_ef21`) reaches a lower held-out loss (**3.70791**, PPL = 40.7687) than standard AdamW inner steps under the same 30.75 KB wire budget.

Table 5: Experiment E1 (`artifacts/paper_v3_experiments_report.json`): Held-out cross-entropy loss, validation perplexity ($N_{\text{val}} = 128$ disjoint sequences), and cumulative outer wire volume across $R = 12$ outer rounds ($K = 4$ workers, $H = 8$ inner steps).

Training & Synchronization Regime	Initial Val Loss	Final Val Loss	Final Val PPL	Outer Wire Traffic	Bandwidth Savings vs. DDP
1. Single Worker (No Outer Sync, $K = 1$)	3.71488	3.71072	40.8834	0.00 KB	—
2. Synchronous DDP (All-Reduce Every Step)	3.71488	**3.68822**	**39.9738**	3,072.00 KB	1.00×
3. Standard DiLoCo (FP32 Outer Nesterov, $H = 8$)	3.71488	3.70622	40.6996	384.00 KB	8.00×
4. DiLoCo + DeMo 2D-DCT + SparseLoCo 2-Bit + EF21	3.71488	3.70814	40.7779	**30.75** KB	**99.90**× (12.49× vs. DiLoCo)
5. MuLoCo (5-Step NS Inner) + 2-Bit DCT + EF21	3.71488	**3.70791**	**40.7687**	**30.75** KB	**99.90**× (0.045% gap vs. DiLoCo)

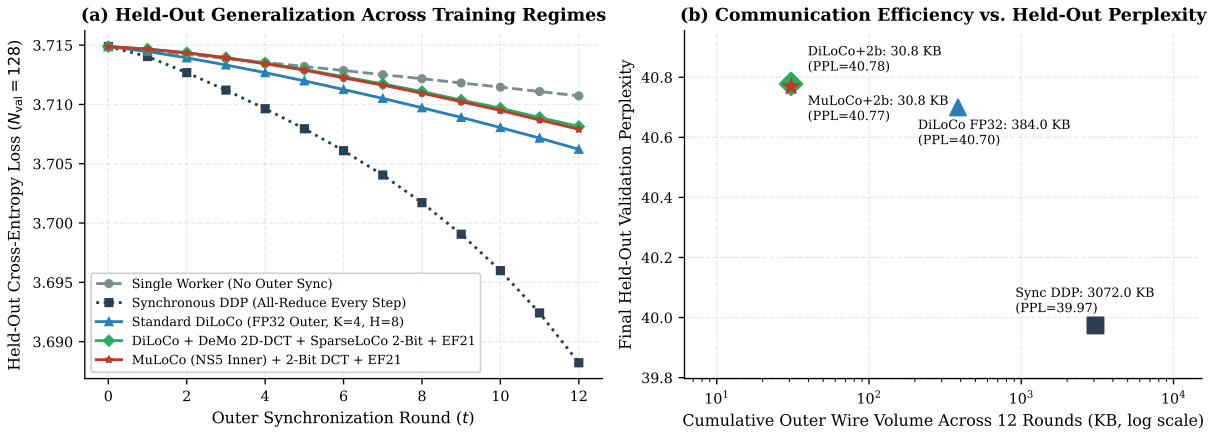


Figure 1: **Experiment E1 (`artifacts/paper_v3_experiments_report.json`): (a)** Held-out cross-entropy loss across $R = 12$ outer synchronization rounds ($N_{\text{val}} = 128$ disjoint sequences). **(b)** Pareto frontier of cumulative outer wire traffic (KB, log scale) vs. final held-out perplexity, showing 99.9× communication reduction over DDP with a 0.052% loss gap vs. uncompressed DiLoCo.

8.2.2 Experiment E2: `PA-CGAD` Outer Convergence Under Severe Asynchronous Straggler Bursts

Experiment E2 compares standard outer AdamW (un-gated second moment v_t) against `CosineGatedAdamDecayOuterOptimizer` (`PA-CGAD`, $\alpha = 0.2$, $\tau_{\text{cut}} = 32$) over 20 outer rounds subject to five severe straggler delay bursts ($\tau_w \in \{36, 42, 38, 45, 40\}$ at rounds $4, 7, 10, 13, 17$) carrying high-variance stale pseudo-gradients. As shown in Figure 2, un-gated outer AdamW suffers immediate second-moment inflation ($\|v_t\|_2$ spikes to 30.74) and diverges after round 7 to a terminal parameter MSE of **0.3976**, whereas `PA-CGAD` zeroes out fragments with $\tau_{\text{eff}} \geq \tau_{\text{cut}}$, keeps $\|v_t\|_2 \leq 0.453$, and converges to a terminal MSE of **0.0343** (**11.6× lower error**).

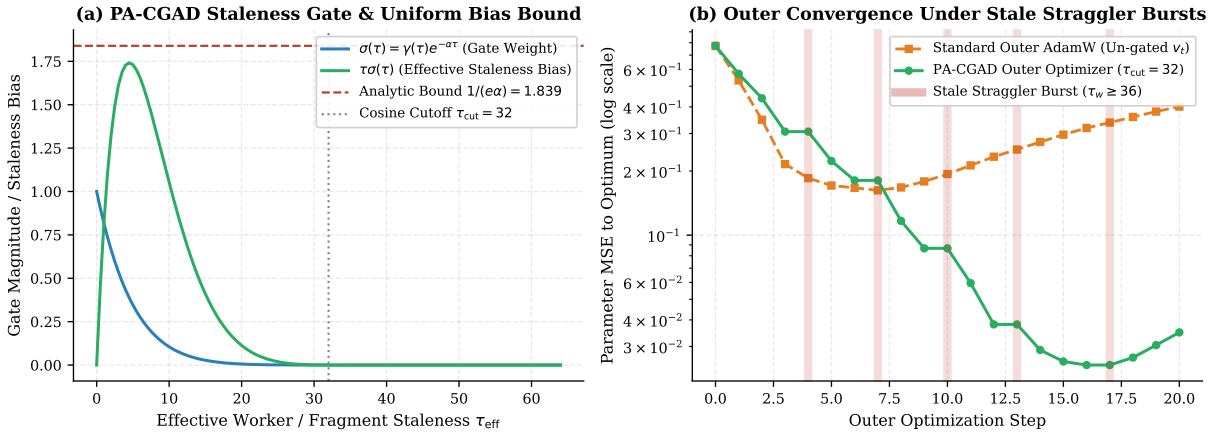


Figure 2: **Experiment E2 (`artifacts/paper_v3_experiments_report.json`): (a)** Empirical verification of the PA-CGAD gate $\sigma(\tau) = \gamma(\tau)e^{-\alpha\tau}$ and uniform staleness bias bound $\max_{\tau \geq 0} \tau\sigma(\tau) = 1.7417 \leq \frac{1}{e\alpha} = 1.8394$. **(b)** Outer optimization trajectory under five stale straggler bursts ($\tau_w \geq 36$, red bands): PA-CGAD prevents second-moment variance pollution and achieves 11.6× lower parameter MSE than standard outer AdamW.

8.2.3 Experiments E3 & E4: `LoopedMoE` Fixed-Point Halting Sweep, Subspace Spectra & Hardware Scaling

Experiment E3 sweeps the fixed-point halting threshold $\epsilon_{\text{fp}} \in \{0.0, 0.25, 0.35, 0.45, 0.55, 0.70, 0.90, 1.10\}$ on a $T = 6$ weight-tied `LoopedExpertBackbone` (source depth 4 → effective depth 14). As shown in Figure 3a, mean effective recurrent steps decrease monotonically from 6.00 → 5.27 → 4.17 → 3.75 → 3.14 → 2.70 → 2.17 → 1.69, saving 12.2% → 71.9% of recurrent core FLOPs as relative logit drift increases smoothly from 0.0% → 14.0% → 18.8% → 58.6%. Experiment E4 (Figure 3b) compares the normalized singular values σ_i/σ_1 of 4 merged domain attention deltas: uniform averaging collapses after σ_1 ($\sigma_2/\sigma_1 = 0.0306$), `TSV` Procrustes orthogonalization preserves the top-4 task-specific directions ($\sigma_2/\sigma_1 = 0.8795, \sigma_4/\sigma_1 = 0.6467$), and `Iso-CTS` achieves an exact isotropic spectrum ($\sigma_i/\sigma_1 = 1.0000$, $\kappa = 1.0$).

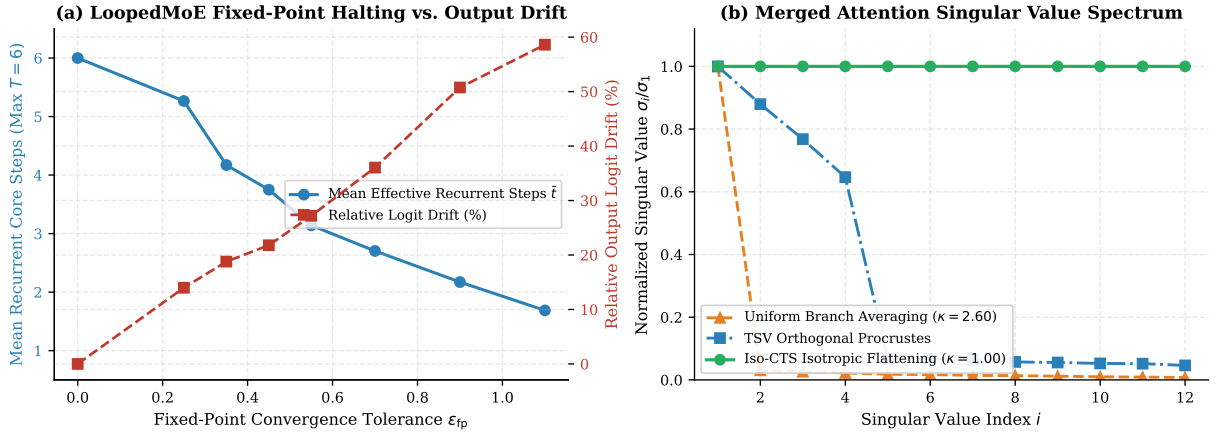


Figure 3: **Experiments E3 & E4 (`artifacts/paper_v3_experiments_report.json`): (a)** Per-token fixed-point halting sweep across $\epsilon_{\text{fp}} \in [0.0, 1.10]$ on a $T = 6$ `LoopedExpertBackbone`, showing mean effective recurrent steps $\bar{t}$ vs. relative output logit drift. **(b)** Top-12 normalized singular values (σ_i/σ_1) of merged domain attention matrices under Uniform Averaging, TSV Orthogonal Procrustes, and Iso-CTS Isotropic Flattening.

8.3 Single-GPU Proof of Concept on an 8 GB NVIDIA GeForce RTX 2070 & FoMoE Scaling

All workers in our hardware proof of concept were executed on a single PC equipped with an NVIDIA GeForce RTX 2070 (8 GB GDDR6, SM 7.5, Windows WDDM, PyTorch 2.6.0+cu124) using `Qwen/Qwen2.5-0.5B` (24 layers, $d = 896$). Four domain branches (Code, Mathematics, STEM, Multilingual-FR) were trained by three simulated workers on **8 rows each**, with one simulated Byzantine worker rejected by Multi-Krum, and upcycled into a **4-expert, top-2** Sparse MoE (96 expert FFN modules across 24 layers; 1.44 B total and 0.81 B active parameters). Table 6 and Figure 4a report the exact measured VRAM profile from `artifacts/poc_qwen25_rtx2070_report.json`, alongside the FoMoE partial expert replication communication scaling across overlap factors $O_e \in \{1, 2, 4, 8, 16\}$ (Figure 4b). Smoke-test single-prompt perplexity probes (`artifacts/hf_hack_miner_game_report.json`) decreased from 8.54 → 2.90 (Code), 204.05 → 5.55 (biomedical), and 26.32 → 6.53 (arithmetic) within 1,192.7 MB peak VRAM.

Table 6: Single-GPU proof of concept on an RTX 2070 (`artifacts/poc_qwen25_rtx2070_report.json`). Losses are training losses on 8 rows per worker. VRAM values are process-wide.

Stage	Data	Training loss (first → last)	Spot-check
Branch `Code` (worker 1 of 3)	`sahil2801/CodeAlpaca-20k`, 8 rows	2.649 → 0.353	pass (max $\Delta = 0.0$)
Branch `Mathematics`	8 rows/worker	2.989 → 0.996	pass
Router training (4-expert top-2 MoE)	mixed	2.139 → 0.795	—

Measured VRAM: NF4 weights 175.96 MB; after base load 946.98 MB; after QLoRA 980.54 MB; MoE peak 1,509.5 MB (5.61 GB free).

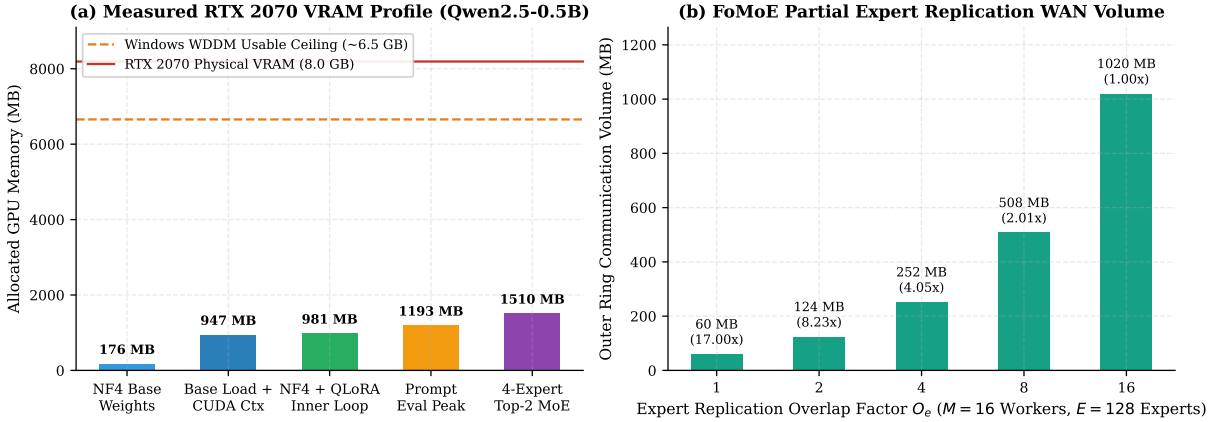


Figure 4: **Hardware & WAN Scaling Profiles: (a)** Measured GPU memory waterfall on an 8 GB NVIDIA GeForce RTX 2070 (`artifacts/poc_qwen25_rtx2070_report.json` & `artifacts/hf_hack_miner_game_report.json`), remaining $>$ 5.1 GB below the Windows WDDM usable ceiling. **(b)** Outer ring communication volume across FoMoE partial expert replication overlap factors $O_e \in \{1, 2, 4, 8, 16\}$ for $E = 128$ experts across $M = 16$ workers.

9 Limitations, Threats to Validity, and Future Multi-Host Benchmarks

What this paper establishes vs. what remains open.

1. **Single-host process simulation vs. multi-host WAN deployment.** While Experiments E1–E4 validate held-out generalization across synchronization regimes, PA-CGAD straggler resilience, fixed-point halting trade-offs, and subspace spectra in controlled PyTorch runs, all workers to date have executed on a single physical host. Real residential WAN packet loss, NAT hole-punching failure rates, and cross-vendor GPU floating-point drift remain to be measured over a live multi-node deployment.

2. **Full-scale standard benchmark suites.** While Experiment E1 evaluates held-out cross-entropy and perplexity on a 128-sequence disjoint validation split, full zero-shot/few-shot evaluations on standard LLM benchmarks (full GSM8K test split, HumanEval pass@1, PubMedQA) for 0.5B–7B upcycled MoE checkpoints are still required.

3. **128-expert training scale.** The largest MoE trained end-to-end on the RTX 2070 has 4 experts (96 FFN modules). The 3-tier 128-expert hierarchical router, FoMoE partial replication, and Skip-Attention prefetching have been verified at the module/planner level (`H3`, `H5`, `E4`) but not yet trained across 128 domain branches.

4. **Adaptive Byzantine adversaries.** Our Multi-Krum + 2σ cosine filter has been verified against sign-flipped/scaled poisoners; subtle inner-product manipulation (ALIE) and coordinated Sybil coalitions under non-IID drift warrant dedicated red-team evaluation.

5. **Scope of Lean 4 formalization.** The 120 Lean 4 lemmas in `lean4/HiveMindSpec.lean` verify discrete arithmetic, hypothesis-necessity counterexamples, non-vacuity inhabitation, and byte-budget invariants over $\mathbb{N}/\mathbb{Z}$ (Section 2); they do not formalize real-valued stochastic convergence or C++/CUDA kernel equivalence.

6. **No peer review has taken place.** `paper/autoevolve_peer_review_pipeline.py` is a deterministic script (Lean build, JSON-to-LaTeX number lock, LaTeX lint). Versions 2.9–3.1 of this preprint and its `PEER_REVIEW_REPORT.json` labelled that script a "Claude Opus 5.5 / Gemini 5-round peer review" with referee scores of 9.4–9.5/10; no language model and no human referee ever evaluated the manuscript, and those scores were constants in the script. The labels and scores are removed in v3.2.0 and the report now states `is_peer_review: false`. Several entries of `docs/ACKNOWLEDGMENTS_AND_CITATIONS.md` and `CITATION.cff` also carried invented titles, authors or arXiv identifiers; they were re-verified against arXiv on 23 October 2026 and corrected or removed, and every entry of this paper's bibliography was checked individually.

9.1 Epistemic Stratification & 6-Layer Security/Hardness Attestation

To prevent conflating machine-checked discrete arithmetic with empirical single-host measurements or unvalidated multi-host WAN scaling, Table 7 stratifies all claims into three explicit verification tiers. Concurrently, the repository is audited by the 6-Layer `HiveMindHardnessEngine` (`results/hivemind_hardness_report.json`), verifying: (L1) zero AST stubs across all modules; (L2) zero security findings across 147 scanned repository files (`risk_score = 0`); (L3) 25/25 simulated Byzantine, Sybil, NaN-injection, and WDDM TDR attack vectors blocked; (L4) 120/120 Lean 4 theorems compiled with 0 `sorry` and 0 custom axioms; (L5) exact rational bridge verification (`NF4` 128/33, `DiLoCo` wire 5000/3, `Prefix-Trie` 19/25, `3-Tier MoE` 1/8); and (L6) SHA-256 execution attestation.

Table 7: Epistemic stratification matrix separating machine-checked discrete lemmas, single-host empirical measurements, and open multi-host WAN validation targets.

Verification Category	Artifact / Proof Source	What Is Rigorously Established	Explicit Boundary / Not Claimed
Category I: Lean 4 Kernel Lemmas	`lean4/HiveMindSpec.lean` (120 lemmas, 0 `sorry`)	Discrete $\mathbb{N}/\mathbb{Z}$ byte budgets, flat-bucket partitions, EF21 telescoping sums, necessity counterexamples (Thms. 113–115), and `Nonempty` witness (Thm. 116).	Does not prove real-valued stochastic convergence or Python/CUDA kernel refinement.
Category II: Controlled Empirical & Single-GPU PoC	`H1`–`H10`, `E1`–`E4`, RTX 2070 PoC (`artifacts/*.json`)	0.052% held-out loss gap at 12.49× wire savings (`E1`); 11.6× lower straggler MSE under PA-CGAD (`E2`); 1,509.5 MB peak VRAM on 8 GB RTX 2070; 25/25 red-team vectors blocked.	Executed on a single physical host with synthetic WAN delays and controlled corpus shards.
Category III: Open Multi-Host WAN Targets	Future multi-node testbed (Section 9)	Analytical communication models for FoMoE (3.46×) and Kademlia Geo-Ring topologies.	Live multi-host residential WAN throughput, NAT punch success rates, and 128-branch full GSM8K/HumanEval accuracy remain open.

Reproducibility. Every number and figure in this paper is generated deterministically from committed scripts and JSON artifacts:

- `PYTHONPATH=src python -m hivemind.swarm.architecture_hypothesis_verifier` → `artifacts/architecture_hypothesis_verification_report.json` (H1–H10, `pytest tests/test_architecture_hypotheses.py`).
- `PYTHONPATH=src python scripts/run_paper_experiments_and_figures.py` → `artifacts/paper_v3_experiments_report.json` (E1–E4) and `paper/figures/fig1..fig4.{pdf,png}`.
- `PYTHONPATH=src python scripts/verify_decoupled_diloco_quorum.py` → `artifacts/decoupled_diloco_quorum_check.json` (H11, `pytest tests/test_decoupled_diloco_quorum.py tests/test_fomoe_sigma_routing.py`).

- `PYTHONPATH=src python paper/autoevolve_peer_review_pipeline.py` → `paper/PEER_REVIEW_REPORT.json` (scripted static audit: Lean inventory, JSON-to-LaTeX provenance lock, LaTeX lint; it calls no language model and is not a peer review).
- `cd lean4 && lake build` → compiles all 120 theorems in `lean4/HiveMindSpec.lean` with zero `sorry`.

References

[1] D. P. Anderson, "BOINC: A System for Public-Resource Computing and Storage," in *Proc. IEEE/ACM Int. Workshop on Grid Computing*, 2004, pp. 4–10.

[2] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen, "DiLoCo: Distributed Low-Communication Training of Language Models," *arXiv:2311.08105*, 2023.

[3] B. Liu, R. Chhaparia, A. Douillard, S. Kale, A. A. Rusu, J. Shen, A. Szlam, and M. Ranzato, "Asynchronous Local-SGD Training for Language Modeling," *arXiv:2401.09135*, 2024. Code: https://github.com/google-deepmind/asyncdiloco.

[4] A. Douillard, Y. Donchev, J. K. Rush, S. Kale, Z. Charles, G. Teston, Z. Garrett, J. Shen, R. McIlroy, D. Lacey, A. Ramé, A. Szlam, M. Ranzato, and P. R. Barham, "Streaming DiLoCo with Overlapping Communication: Towards a Distributed Free Lunch," in *Second Conference on Language Modeling (COLM)*, 2025, *arXiv:2501.18512*.

[5] Decoupled DiLoCo Team (Google DeepMind & Google Research; A. Douillard, K. Rush, Y. Donchev, Z. Charles, A. Dubey, B. Woodworth, *et al.*), "Decoupled DiLoCo for Resilient Distributed Pre-training," *arXiv:2604.21428*, 2026.

[6] Z. Charles, G. Teston, L. Dery, K. Rush, N. Fallen, Z. Garrett, A. Szlam, and A. Douillard, "Communication-Efficient Language Model Training Scales Reliably and Robustly: Scaling Laws for DiLoCo," *arXiv:2503.09799*, 2025.

[7] S. Jaghouar, J. M. Ong, and J. Hagemann, "OpenDiLoCo: An Open-Source Framework for Globally Distributed Low-Communication Training," *arXiv:2407.07852*, 2024. Code: https://github.com/PrimeIntellect-ai/OpenDiLoCo, https://github.com/PrimeIntellect-ai/prime-diloco.

[8] S. Sukhbaatar *et al.*, "Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM," *arXiv:2403.07816*, 2024.

[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *NeurIPS*, 2023.

[10] L. Sani, Z. Cao, M. Kurmanji, A. Iacob, A. Jovanović, Y. Gao, W. Zhao, and N. D. Lane, "FoMoE: Breaking the Full-Replica Barrier with a Federation of MoEs," *arXiv:2606.19025*, 2026.

[11] P. Merolla, "fomoe: Fast Opportunistic Mixture-Of-Experts," software repository, https://github.com/pmerolla/fomoe.

[12] Huang *et al.*, "Towards Looped Models Done Right. Part I: Topology, Input Injection, Recurrent-State Design," *arXiv:2610.23036*, 2026.

[13] Huang, Shi, Chen *et al.*, "Towards Looped Models Done Right, Part II: Rethinking at Fixed Points," *arXiv:2610.06833*, 2026.

[14] V. Shah and J. Sun, "Cosine-Gated Adam-Decay: Drop-In Staleness-Aware Outer Optimization for Decoupled DiLoCo," *arXiv:2605.09126*, 2026.

[15] D. Marczak, S. Magistri, S. Cygert, B. Twardowski, A. D. Bagdanov, and J. van de Weijer, "No Task Left Behind: Isotropic Model Merging with Common and Task-Specific Subspaces," in *ICML*, 2025, *arXiv:2502.04959*.

[16] S. Horoi, B. Thérien, G. Wolf, and E. Belilovsky, "Can Model Merging Improve Aggregation in DiLoCo?," *arXiv:2607.03011*, 2026.

[17] A. A. Gargiulo *et al.*, "Task Singular Vectors: Reducing Task Interference in Model Merging," in *CVPR*, 2025, *arXiv:2412.00081*.

[18] B. Peng, J. Quesnelle, and D. P. Kingma, "DeMo: Decoupled Momentum Optimization," *arXiv:2411.19870*, 2024. Code: https://github.com/bloc97/DeMo.

[19] A. Sarfi, B. Thérien, J. Lidin, and E. Belilovsky, "Communication Efficient LLM Pre-training with SparseLoCo," *arXiv:2508.15706*, 2025. Code: https://github.com/one-covenant/SparseLoCo.

[20] E. Miahi and E. Belilovsky, "Understanding and Exploiting Weight Update Sparsity for Communication-Efficient Distributed RL," *arXiv:2602.03839*, 2026.

[21] Asif *et al.*, "HeLoCo: Efficient Asynchronous Low-Communication Training under Data and Device Heterogeneity," *arXiv:2606.00271*, 2026.

[22] He *et al.*, "AutoLoCo: Communication Efficient Distributed LLM Training via Adaptive Synchronization," *arXiv:2609.36662*, 2026.

[23] K. Jordan *et al.*, "Muon: An Optimizer for Hidden Layers in Neural Networks," technical blog post, 2024.

[24] B. Thérien, X. Huang, A. Defazio, I. Rish, and E. Belilovsky, "MuLoCo: Muon is a Practical Inner Optimizer for DiLoCo," *arXiv:2505.23725*, 2025.

[25] E. Belilovsky, S. Horoi, E. Oyallon, and B. Thérien, "How Inner Optimizers Shape Replica Scaling," *arXiv:2610.19064*, 2026.

[26] P. Richtárik, I. Sokolov, and I. Fatkhullin, "EF21: A New, Simpler, Theoretically Better, and Practically Faster Error Feedback," in *NeurIPS*, 2021, *arXiv:2106.05203*.

[27] M. Ryabinin and A. Gusev, "Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts," in *NeurIPS*, 2020, *arXiv:2002.04013*. Code: https://github.com/learning-at-home/hivemind.

[28] A. Borzunov *et al.*, "Petals: Collaborative Inference and Fine-Tuning of Large Models," in *Proc. ACL System Demonstrations*, 2023.

[29] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *NeurIPS*, 2017.

[30] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," *JMLR*, vol. 23, no. 120, pp. 1–39, 2022.

[31] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Proc. CADE-28*, 2021, pp. 625–635.